

PRODUCT ASSIGNMENT[DP PROBLEMS]

UDAY CHIRRA

2303A510G2

1. Delete and Earn

```
class Solution {
    public int deleteAndEarn(int[] nums) {
        int max = 0;

        for (int x : nums)
            max = Math.max(max, x);

        int[] sum = new int[max + 1];

        for (int x : nums)
            sum[x] += x;

        int prev = 0;
        int curr = 0;

        for (int i = 1; i <= max; i++) {
            int next = Math.max(curr, prev + sum[i]);
            prev = curr;
            curr = next;
        }

        return curr;
    }
}
```

2. Partition Equal Subset Sum

```
class Solution {
    public boolean canPartition(int[] nums) {
        int sum = 0;

        for (int x : nums)
            sum += x;
    }
}
```

```

    if (sum % 2 != 0)
        return false;

    int target = sum / 2;
    boolean[] dp = new boolean[target + 1];
    dp[0] = true;

    for (int x : nums) {
        for (int j = target; j >= x; j--) {
            dp[j] = dp[j] || dp[j - x];
        }
    }

    return dp[target];
}

```

3. Target Sum

```

class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int sum = 0;

        for (int x : nums)
            sum += x;

        if (Math.abs(target) > sum || (sum + target) % 2 != 0)
            return 0;

        int required = (sum + target) / 2;
        int[] dp = new int[required + 1];
        dp[0] = 1;

        for (int x : nums) {
            for (int j = required; j >= x; j--) {
                dp[j] += dp[j - x];
            }
        }
    }
}

```

```

        return dp[required];
    }
}

```

4. Minimum Cost For Tickets

```

class Solution {
    public int mincostTickets(int[] days, int[] costs) {
        int[] dp = new int[366];
        boolean[] travel = new boolean[366];

        for (int day : days)
            travel[day] = true;

        for (int day = 1; day <= 365; day++) {
            if (!travel[day]) {
                dp[day] = dp[day - 1];
            } else {
                int one = dp[day - 1] + costs[0];
                int seven = dp[Math.max(0, day - 7)] + costs[1];
                int thirty = dp[Math.max(0, day - 30)] + costs[2];

                dp[day] = Math.min(one, Math.min(seven, thirty));
            }
        }

        return dp[365];
    }
}

```

7. Basketball Exercise

```

import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        long[][] a = new long[2][n];
    }
}

```

```

for (int i = 0; i < n; i++)
    a[0][i] = sc.nextLong();

for (int i = 0; i < n; i++)
    a[1][i] = sc.nextLong();

long[][] dp = new long[2][n];

dp[0][0] = a[0][0];
dp[1][0] = a[1][0];

for (int i = 1; i < n; i++) {
    dp[0][i] = a[0][i] + dp[1][i - 1];
    dp[1][i] = a[1][i] + dp[0][i - 1];

    if (i > 1) {
        dp[0][i] = Math.max(dp[0][i], a[0][i] + dp[1][i - 2]);
        dp[1][i] = Math.max(dp[1][i], a[1][i] + dp[0][i - 2]);
    }
}

System.out.println(Math.max(dp[0][n - 1], dp[1][n - 1]));
}
}

```

8. Petr and a Combination Lock

```

import java.util.*;

class Main {
    static int n;
    static int[] a;

    static boolean solve(int index, int angle) {
        if (index == n)
            return angle % 360 == 0;

        return solve(index + 1, angle + a[index]) ||
            solve(index + 1, angle - a[index]);
    }
}

```

```
}  
  
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
  
    n = sc.nextInt();  
    a = new int[n];  
  
    for (int i = 0; i < n; i++)  
        a[i] = sc.nextInt();  
  
    System.out.println(solve(0, 0) ? "YES" : "NO");  
}  
}
```